



Introduction to Data Structures

Renis Ndjegoni



ARGUMENT

In these pages, I simply sought a different way to take notes, moving beyond the traditional pen & pencil. I discovered this technology & asked myself: why not use it to create art? Imagine doing beautiful work, not just taking notes, but feeling it. Why not? I have always liked art, especially the scenery art you can write with, & the architecture of all the artists who contributed in Europe to religion, showing how beautifully people dedicated themselves to CHRISTIANITY through Art. I honor the ancestors who taught us that beauty is truth—in their books, their cathedrals, & their paintings. Let us, therefore, reunite this sacred aesthetic; let us clothe the logic of code in the eternal glory of Art.

THE ALGORITHM 1. can you construct anything, on a

1. First, you will see how Data Structures are represented visually through animations. Animations are an essential part of the learning experience, accompanied by simple explanations.

¹Now we are looking for the most important features & firstly to explain how we proceed, because we desire to create something great. We need to understand the essence of Algorithms & Data Structures, for what purpose we utilize them, & the way to become a Master in Algorithms.



ALGORITHMS are a common subject in Computer Science Engineering. As a definition, it is commonly stated: *It is a step-by-step procedure for solving a problem*. But what is a program? A program is the machine that employs these algorithms to achieve its goals.

BUT WHAT IS THE DIFFERENCE BETWEEN A PROGRAM & AN ALGORITHM?

2. We will learn how to code various Data Structures with step-by-step instructions. We will go over working source code, coding exercises, and multiple-choice questions to ensure hands-on experience.

Let us see the first difference: the Software Life Cycle. There are two phases of project development; two important phases: the Design phase & the Implementation phase.

Whatever you wish to manufacture or construct through engineering, the most important step is Designing. First, you must design; make your design perfect so you can understand what you are going to construct & what you are going to develop. You cannot develop anything, nor

trial-and-error basis.

If you construct something & it turns out wrong or is destroyed, & you have to create a new one—that is WRONG. It is wasting time. In programming, the point is to Design first, & Code after.

3. We begin with the fundamentals: Dynamic Array, Linked List, Stack, and Queue. Then, we move to intermediate structures: Heap-based Priority Queue, Union Find, and Binary Trees.

So, algorithms are written at ^aDESIGN time, & programs are written at ^bIMPLEMENTATION time. First, you create a design of what your procedures & software are going to do.

The person who writes algorithms should have ^cDOMAIN KNOWLEDGE, whereas programs are written by ^dPROGRAMMERS.

To write algorithms, you can use ^eANY LANGUAGE (even Natural Language), while programs are written using a specific ^fPROGRAMMING LANGUAGE (like C/C++). & if it is efficient in terms of ^gTIME & SPACE. For programs, we perform ^hTESTING.

Algorithms are independent of ⁱHARDWARE & OS; they do not

4. Finally, the Advanced section covers Hash Tables with collision resolution techniques, and the Fenwick Tree, which boasts efficiency in Range Sum Queries.

indicate the machine used. However, when we write a program, it is dependent on the ^j**HARDWARE** we use.

- 5 After writing an algorithm, we
^k**ANALYZE** it to verify if we are achieving the goal